Lezione 7 - Sistemi REST

Introduzione

Nelle lezioni precedenti abbiamo analizzato i meccanismi fondamentali alla base della comunicazione tra client e server: il funzionamento del protocollo HTTP, la struttura delle richieste e delle risposte, il ruolo degli header e del body, fino ad arrivare al formato JSON come standard moderno per lo scambio dei dati.
Abbiamo inoltre introdotto il concetto di API e visto come queste espongano risorse accessibili tramite endpoint ben definiti, utilizzando payload strutturati per trasmettere informazioni in modo chiaro e interoperabile.

Il passo successivo consiste nel comprendere **come organizzare** questi elementi in un vero e proprio modello architetturale.
In altre parole: quando progettiamo un sistema basato su API, quali regole seguiamo per rendere la comunicazione semplice, prevedibile e scalabile?

A questa esigenza risponde **[[Lezione 6 - API#•** REST (REpresentational State Transfer)**|REST (REpresentational State Transfer)]]**:

- **uno stile architetturale che si è affermato come lo standard de facto per la realizzazione delle API web moderne.**
  REST fornisce un insieme di principi e convenzioni che guidano la progettazione di servizi web facilmente utilizzabili, interoperabili e aderenti ai meccanismi naturali del protocollo HTTP.

Sistemi REST

REST (*REpresentational State Transfer*) è uno stile architetturale basato su un insieme di vincoli, linee guida e buone pratiche pensate per progettare sistemi distribuiti semplici da usare, scalabili e coerenti.
È importante chiarire sin da subito che **REST non è un'architettura completa, né una tecnologia specifica.**
Non è un protocollo, non è un framework, non è una libreria:

- **è un modello di progettazione che definisce come strutturare un sistema affinché sia prevedibile, chiaro e soprattutto interoperabile.**

L'obiettivo principale di REST è offrire un riferimento comune per evitare che ogni sviluppatore implementi API a modo proprio, generando sistemi

incoerenti, difficili da integrare e complicati da mantenere nel tempo.
Quando un servizio rispetta i vincoli e i principi di questo modello, si dice che è un sistema **RESTful**.

Note

Origini e contesto storico

L'idea di REST nasce nel 2000 all'interno della tesi di dottorato di **Roy Fielding**, intitolata *Architectural Styles and the Design of Network-based Software Architectures*. Quell'anno rappresentava un momento cruciale per l'evoluzione del web:

- Internet cresceva rapidamente a livello globale
- Le applicazioni distribuite diventavano sempre più complesse
- Le tecnologie disponibili erano spesso pesanti, rigide e poco intuitive
- Il web stesso dimostrava che semplicità e standardizzazione potevano ottenere risultati enormi

In questo scenario, Fielding analizzò i punti di forza del web e cercò di formalizzarli in un modello chiaro e generale, capace di guidare la progettazione dei futuri sistemi distribuiti. Così nacque REST.

Chi era Roy Fielding

Fielding non era soltanto uno studioso, ma uno dei principali protagonisti dell'evoluzione del web. Tra i suoi contributi più importanti:

- co-fondatore del progetto **Apache HTTP Server**, che per anni è stato il web server più utilizzato al mondo
- autore principale delle specifiche **HTTP/1.0** e **HTTP/1.1**
- co-fondatore della **Apache Software Foundation**
- membro attivo del **W3C**, il consorzio che definisce gli standard del web

Questa posizione privilegiata gli permise di osservare contemporaneamente sia le difficoltà reali degli sviluppatori sia i limiti teorici dei modelli distribuiti dell'epoca.

Prima di REST: un panorama frammentato e complesso

Prima del 2000, la realizzazione di sistemi distribuiti era spesso un percorso tortuoso. Tra le tecnologie più utilizzate:

- **CORBA**: **molto potente, ma estremamente complessa e difficile da manutenere**
- **RPC (Remote Procedure Call)**: **cercava di nascondere la natura distribuita, provocando problemi di affidabilità**
- **SOAP**: **formalmente rigoroso, ma eccessivamente verboso e basato su messaggi XML pesanti**
- Tecnologie proprietarie: ogni azienda implementava protocolli personalizzati, incompatibili tra loro

Il risultato era un ecosistema caotico:

- servizi poco interoperabili, scarsa scalabilità, implementazioni difficili da integrare e sistemi in cui client e server erano troppo strettamente legati.

I problemi che REST voleva risolvere

REST nacque quindi con l'obiettivo di affrontare alcune criticità ricorrenti nei sistemi distribuiti:

- **eccessiva complessità delle tecnologie dell'epoca**
- **accoppiamento troppo stretto tra client e server**
- **scarsa scalabilità, soprattutto sotto carichi elevati**
- **difficoltà di interoperabilità tra sistemi diversi**
- **mancanza di un modello condiviso per la progettazione delle API**

Analizzando ciò che aveva reso il web così efficace, Fielding estrasse una serie di principi che – se applicati correttamente – avrebbero permesso di creare sistemi semplici, prevedibili e facili da integrare.
Da qui nasce REST, lo stile architetturale che oggi costituisce la base della maggior parte delle API web moderne.

Le motivazione fondamentali di REST

Alla luce di ciò le motivazioni fondamentali delle REST sono:

1. Semplicità

Il principio guida di REST è eliminare la complessità superflua.
Invece di inventare un nuovo protocollo di comunicazione, REST utilizza l'**HTTP**, che già governa il web, con vantaggi significativi:

✓ **La semplicità di REST**

- **Curva di apprendimento ridotta**:
  - **Gli sviluppatori conoscono già i verbi HTTP (GET, POST, PUT, DELETE, ecc.) e i codici di stato (200, 404, 500, ecc.), senza dover apprendere nuovi standard complessi.**
- **Niente "buste" complicate**:
  - **A differenza di protocolli come SOAP, che avvolgono ogni messaggio in strutture XML complesse, le richieste REST sono dirette e leggibili.**
- **Debugging facilitato**:
  - **Le richieste REST sono semplici messaggi di testo che possono essere facilmente ispezionati con strumenti di sviluppo di qualsiasi browser.**

Esempio: Una richiesta REST GET a `/utenti/123` è immediatamente comprensibile, mentre in SOAP sarebbe necessario costruire un intero documento XML.

2. Scalabilità

REST è progettato per gestire sistemi con alto numero di utenti e richieste simultanee.
Alcuni principi chiave che permettono questa scalabilità sono:

✓ **Principi chiave**

- **Stateless (senza stato)**:
  - **Ogni richiesta contiene tutte le informazioni necessarie per essere elaborata. Il server non mantiene alcuna memoria dello stato del client tra una richiesta e l'altra. Questo permette di distribuire il carico su più server senza problemi.**
- **Caching**:
  - **Le risposte possono essere memorizzate temporaneamente (client o nodi intermedi) riducendo il numero di richieste al**

server e migliorando le prestazioni.

- **Sistema a livelli (Layered System)**:
  - **Tra client e server possono esserci più livelli intermedi, come proxy o bilanciatori di carico, senza che client o server debbano modificarne il comportamento.**

3. Indipendenza

REST favorisce disaccoppiamento e interoperabilità tra client e server:

✓ **Caratteristiche**

- **Indipendenza di piattaforma e linguaggio**:
  - **Un client JavaScript può comunicare con un server Python, Java o qualsiasi altra tecnologia senza problemi.**
- **Evoluzione indipendente**:
  - **Client e server possono essere aggiornati separatamente, purché il contratto dell'API non venga rotto.**

Questi vincoli garantiscono che un'architettura RESTful sia **scalabile, semplice e affidabile**, proprio come il web stesso.

Vincolo "Stateless" (Senza Stato) e Risorse

I sistemi REST si basano su un insieme di vincoli pensati per garantire:

- **scalabilità**,
- **semplicità**
- **affidabilità**,
  seguendo gli stessi principi che hanno reso il web un sistema robusto.
  Tra questi vincoli, uno dei più importanti è quello della **statelessness**.

1. Vincolo Statelessness

- **Significato**:
  - **Un servizio è stateless quando ogni richiesta inviata al server contiene già tutte le informazioni necessarie per essere capita ed elaborata, senza che il server debba ricordare nulla di quanto avvenuto in precedenza.**

- Di conseguenza non vi è **nessuna sessione sul server**:
  - **Il server non mantiene sessioni né stato tra una richiesta e la successiva**.
  - **Se il client deve dimostrare la propria identità o passare parametri, deve farlo ogni volta, includendo le informazioni nella richiesta.**

✓ **Done**

Perché è fondamentale?

- **Scalabilità**:
  - **Qualsiasi server dell'infrastruttura può gestire qualsiasi richiesta, favorendo il bilanciamento del carico e semplificando l'aggiunta di nuovi nodi.**
- **Affidabilità**:
  - **Se un server ha un problema, la richiesta può essere ripetuta verso un altro server, senza perdere informazioni sul contesto.**
- **Semplicità**:
  - **Il server non deve gestire sessioni o memorie temporanee, riducendo complessità e possibilità di errore.**

Analogia della pizzeria:

Immagina una pizzeria in cui **non esiste un cameriere dedicato a un determinato tavolo**.
Tutti i camerieri possono servire chiunque, perché ogni ordine contiene già tutto ciò che serve per essere preparato.

✓ **Vantaggio: il lavoro si distribuisce facilmente.**

× **Svantaggio: nessun cameriere conosce preferenze o storicità del cliente; ogni ordine è indipendente.**

2. Interazione basata su Risorse

REST organizza il sistema attorno al concetto di **risorsa:**

- **ciascuna identificata da un URL unico.**

Le operazioni sulle risorse seguono i verbi standard di HTTP, che conferiscono coerenza e prevedibilità all'interfaccia.

- `GET /utenti/123` → Recupera la risorsa "utente 123"
- `POST /utenti` → Crea una nuova risorsa
- `PUT /utenti/123` → Sostituisce completamente la risorsa esistente
- `DELETE /utenti/123` → Elimina la risorsa

Questa combinazione di **URL + verbi HTTP** rende l'interazione semplice, intuitiva e altamente standardizzata.

Architettura Stateful (Con Stato)

Per comprendere meglio il vantaggio del modello stateless, è utile confrontarlo con l'approccio tradizionale **stateful**, dove il server mantiene un **contesto** per ogni client.

In un'architettura **stateful**:

- **il server crea e mantiene una sessione dedicata per ciascun client.**
- **Questo significa che il server "ricorda" chi è il client e tiene traccia di tutte le sue interazioni precedenti.**

Analogia: la pizzeria con cameriere personale

immagina una pizzeria di lusso dove ogni cliente ha **un cameriere personale** per tutta la durata della cena.

✓ Vantaggio:

il cameriere conosce le preferenze del cliente ("il solito?") e lo stato preciso del suo ordine, offrendo un servizio più personalizzato e immediato.

× Svantaggio:

modificare l'ordine può essere più complicato se non si passa dal cameriere assegnato.
Se arrivano molti clienti contemporaneamente, il carico di lavoro non si

distribuisce facilmente, perché ogni cameriere è legato ai propri tavoli. Inoltre, se il cameriere assegnato si assenta, la continuità del servizio si interrompe.

Vantaggi principali:

- **Performance**:
  - **le richieste possono essere più leggere, poiché il server conosce già il contesto e lo stato del client.**
- **Esperienza utente**:
  - **l'interazione risulta più fluida e contestualizzata, simile a una conversazione continua.**

Svantaggi principali:

- **Scalabilità limitata**:
  - **è più difficile distribuire il carico su più server, perché ciascun client è legato a un server specifico.**
  - **Reindirizzare la richiesta a un altro server può causare problemi, in quanto quest'ultimo non conosce lo stato della sessione.**
- **Maggiore complessità e costi**:
  - **gestire sessioni dedicate aumenta la complessità del server e rendere scalabile l'infrastruttura richiede più risorse.**
- **Fragilità**:
  - **se il server che mantiene la sessione si guasta, tutte le informazioni sullo stato dei client vanno perse, costringendo l'utente a ricominciare da capo**

Architettura Stateless vs Stateful

- **Stateless (senza stato)**:
  - **Il server non mantiene informazioni sul client tra le richieste.**
  - Vantaggi:
    - **scalabilità,**
    - **affidabilità,**
    - **facilità di monitoraggio**.
  - Svantaggi:

    - **il client deve inviare dati aggiuntivi (es. token) ad ogni richiesta.**
- **Stateful (con stato)**:
  - **Il server mantiene una sessione per ogni client.**
  - Vantaggi:
    - **migliore performance**
    - **esperienza utente personalizzata**.
  - Svantaggi:
    - **scalabilità limitata,**
    - **complessità maggiore**
    - **fragilità in caso di guasti**.

Analogia della pizzeria con camerieri dedicati:

Quindi ritornando all'analogia della pizzeria in un sistema stateless:
Ogni cliente non ha un cameriere personale.
Mentre in un sistema stateful:
ogni cliente ha un cameriere personale.

✓ **Vantaggi: servizio più personalizzato.**

× **Svantaggi: difficoltà nel gestire grandi numeri di clienti e carichi variabili.**

Faq

Perché i Servizi Stateless Sono Più Convenienti ed Efficienti

Uno dei motivi principali per cui i sistemi REST hanno avuto così grande successo è il **vantaggio economico** associato al modello **stateless**.

Nei sistemi **[[Lezione 7 - Sistemi REST#**Architettura Stateful (Con Stato)**|stateful]]**, ogni volta che un client interagisce con un server: **quest'ultimo deve mantenere una sessione attiva, cioè uno spazio di memoria dedicato in cui conservare informazioni sullo stato della conversazione (dati parziali, preferenze, step già completati, ecc.).**
Questa gestione dello stato, però, ha un costo molto elevato:

- **ogni sessione occupa memoria sul server,**
- **più client si collegano, più sessioni devono essere gestite,**
- **per scalare è necessario aggiungere server più potenti o nuovi nodi, con costi crescenti.**

Con l'arrivo dei sistemi REST la logica cambia completamente:

- **la sessione lato server non è più necessaria**.
  Il servizio diventa **stateless**: **cioè ogni richiesta contiene già tutte le informazioni necessarie per essere processata, senza dover dipendere da una memoria condivisa precedente.**
  Questo approccio porta a due conseguenze fondamentali:

1. Riduzione dei costi e aumento della scalabilità:

   Poiché il server non deve più mantenere sessioni dedicate:
   - **si riduce l'uso di memoria e risorse.**
   - **diventa molto più semplice distribuire il carico tra più server.**
   - **ogni richiesta può essere gestita da qualsiasi nodo dell'infrastruttura.**

Il risultato è una scalabilità **più economica, lineare e affidabile**.

2. Disaccoppiamento totale tra Client e Server

   Nei servizi stateless, la gestione della "conversazione" viene spostata sul **client**, che è un programma eseguito su una macchina indipendente.

   Questo significa che:
   - **le operazioni su più step (wizard, procedure multistep, form complessi, ecc.) vengono gestite temporaneamente dal client.**
   - **solo al termine di tutto il processo il client invia una chiamata completa al server, che la elabora senza dover conoscere gli step precedenti.**

In questo modo, chi sviluppa il servizio lato server non deve preoccuparsi di come il client gestisce la sua interazione interna: può concentrarsi solo sulla logica della singola richiesta.

La Scala di Maturità REST (Richardson Maturity Model)

Roy Fielding, nel definire i principi architetturali dello stile REST, ha anche ispirato l'elaborazione di un modello che permette di classificare il grado di aderenza di un sistema a tali principi.
Questo modello è conosciuto come **Scala di Maturità di Richardson** (Richardson Maturity Model):

- **suddivide le API in diversi livelli, dal più rudimentale al pienamente RESTful**.

È importante sottolineare che non si tratta di una norma o di una legge formale:

- **è semplicemente uno strumento utile per comprendere quanto profondamente un'API rispetti lo stile REST e per valutare in quale direzione evolvere un sistema esistente.**

Livello 0 – Il "Far West"

Al livello zero troviamo i sistemi con il più basso grado di maturità.
In questa categoria rientrano le API che trattano l'interfaccia remota come un unico punto di accesso:

- **spesso utilizzando un solo** endpoint **generico (ad esempio** `POST /api`**) per eseguire qualunque operazione.**
  In questi sistemi **non esistono vere e proprie risorse**, né una differenziazione tra operazioni basate sui metodi HTTP.
  I vincoli REST sono completamente assenti e coesistono stili architetturali molto diversi, da qui il nome "Far West".

Livello 1 – Risorse

Il primo passo verso REST consiste nell'introduzione del concetto di **risorsa**:

- **rappresentata tramite URI specifici.**

A questo livello l'API dispone di endpoint distinti per gli elementi del dominio (ad esempio `/clienti`, `/ordini`, `/prodotti`).
Tuttavia, pur iniziando a separare le entità, **le operazioni sono ancora accentrate**:

- **spesso tutte le interazioni avvengono tramite un unico metodo HTTP (di solito `POST`).**
  Il sistema migliora in organizzazione, ma non sfrutta ancora la semantica del protocollo HTTP.

Livello 2 – Verbi HTTP e Codici di Stato

Il livello 2 introduce un elemento fondamentale dello stile REST:

- la **semantica dei metodi HTTP**.
  Ogni operazione viene espressa tramite il verbo più appropriato:
- **`GET` per recuperare risorse**
- **`POST` per crearle**
- **`PUT` o `PATCH` per aggiornarle**
- **`DELETE` per eliminarle**

Inoltre, il sistema utilizza i codici di stato HTTP per comunicare l'esito delle operazioni (ad esempio `200 OK`, `404 Not Found`, `201 Created`).
Questo livello rappresenta nella pratica il traguardo raggiunto dalla maggior parte delle API moderne ed è già sufficiente per ottenere un'interfaccia ben progettata, chiara e comprensibile.

Livello 3 – HATEOAS (Hypermedia as the Engine of Application State)

Il livello più alto della scala introduce il vincolo dell'**hypermedia**.
In un sistema di livello 3:

- **ogni risorsa non contiene soltanto dati, ma anche link che guidano il client attraverso le possibili azioni successive.**

In altre parole, lo stato dell'applicazione è determinato anche dai collegamenti ipermediali forniti dal server:

- **l'interazione diventa auto-documentata e dinamica.**

Si tratta del livello che più si avvicina all'architettura REST completa teorizzata da Fielding.
Tuttavia, non sempre è necessario o conveniente implementarlo integralmente:
molte API si fermano al livello 2 pur essendo progettate in modo efficace.

Approfondiamo ogni livello

Livello 0 – La “Palude” del POX (The Swamp of POX)

Il Livello 0 rappresenta il punto di partenza della Scala di Maturità REST:

- **è il gradino più basso, quello in cui un sistema utilizza il** protocollo HTTP **nel modo più semplice (e più limitato) possibile.**
- L’acronimo **POX** significa ***Plain Old XML*** – anche se oggi potremmo parlare indifferentemente di *Plain Old JSON* – e **descrive un approccio in cui il server e il client si scambiano semplici messaggi strutturati senza alcuna attenzione ai principi del web.**

Come funziona un sistema POX

A questo livello, **l’HTTP non viene sfruttato come protocollo applicativo dotato di semantica propria, ma viene trattato come un semplice “tunnel” per invocare funzioni remote, in stile RPC (Remote Procedure Call).**
Le caratteristiche fondamentali sono:

- **Un unico endpoint**:
  - **esiste un solo URL che riceve tutte le richieste, indipendentemente dall’operazione da eseguire.**
- **Uso quasi esclusivo del metodo POST**:
  - **che si tratti di leggere, creare o eliminare dati, le richieste vengono inviate sempre allo stesso modo.**
- **Assenza del concetto di risorsa**:
  - ==il sistema non espone elementi del dominio tramite URI
  - significativi; lavora invece su “funzioni” o “comandi”.
- **Messaggi sincroni**:
  - **client e server comunicano tramite scambio immediato di dati via HTTP, senza alcun meccanismo di navigazione tra stati.**

Il risultato è un’API che “parla” con il server, ma che non utilizza nessuno dei vincoli dello stile REST né la struttura naturale del web.

ⓘ Perché si parla di “palude”

La metafora della palude (*swamp*) vuole evidenziare che questo approccio tende a diventare presto poco chiaro, confuso e difficile da scalare.
L’assenza di risorse, l’uso indistinto dei metodi HTTP e la concentrazione di

tutte le operazioni in un unico endpoint portano a un codice meno leggibile, meno standard e più fragile.

In pratica, l'API funziona… ma senza alcun ordine architetturale.

Esempio pratico

Immaginiamo un sistema che deve gestire utenti.
Una tipica API di Livello 0 espone un solo endpoint per tutte le operazioni:

```
https://api.example.com/userService

```

A questo URL arrivano tutte le richieste, indipendentemente dall'azione da eseguire.

1. **Richiedere i dati di un utente**
   Anche per una semplice operazione di lettura, il client invia un POST. L'intenzione ("getUser") è indicata nel payload.
2. **Creare un nuovo utente**
   La richiesta è identica nella forma: sempre un POST allo stesso URL. Cambia solo l'azione dichiarata nel corpo (ad esempio "createUser").
3. **Eliminare un utente**
   Di nuovo un POST allo stesso endpoint, con un payload che indica `"deleteUser"`.

Il server interpreta il contenuto del messaggio e decide quale operazione svolgere.
In altre parole, **la logica dell'API non è espressa nell'URI o nei metodi HTTP, ma nascosta nel corpo della richiesta.**

Caratteristiche principali dei sistemi POX

I sistemi che rientrano nel Livello 0 condividono alcune caratteristiche strutturali molto semplici, che mostrano chiaramente come non sfruttino le potenzialità del web:

- **Comunicazione sincrona tramite HTTP:**

    - **il protocollo viene usato solo come mezzo di trasporto per scambiare messaggi. Nessuna delle sue funzionalità applicative viene realmente valorizzata.**
- **Assenza del concetto di risorsa:**
    - **invece di esporre elementi del dominio tramite URL significativi, il sistema lavora con "funzioni" o "operazioni" remote, come in un classico modello RPC.**
- **Uso minimo dei verbi HTTP:**
    - **tipicamente l'interazione si riduce a GET e soprattutto POST, utilizzati indistintamente per qualsiasi tipo di operazione (lettura, creazione, cancellazione).**

Questi aspetti rendono il sistema funzionante, ma poco chiaro e scarsamente strutturato.

Esempio pratico del Livello 0

Per capire come si comporta un'API POX, immaginiamo un sistema che gestisce utenti.
Un'API di Livello 0 espone un unico endpoint, responsabile di tutte le operazioni:

Plain text

```
https://api.example.com/userService
```

Il client invia sempre lo stesso tipo di richiesta e indica l'azione da eseguire all'interno del payload.

1. **Richiedere i dati di un utente:**
    - **Anche se è un'operazione di sola lettura, si utilizza comunque il metodo POST.**
    - **Nel corpo della richiesta viene indicata l'azione, ad esempio `"getUser"`.**

JSON

```
POST /useService
Host: api.example.com
Content-Type: application/json
{
```

```
    "action": "getUser",
    "userId": 123
}
```

2. **Creare un nuovo utente:**
   - **La struttura della richiesta non cambia: sempre un POST allo stesso URL.**
   - **Nel payload compare l'istruzione `"createUser"` e i dati necessari alla creazione.**

```JSON
POST /useService
Host: api.example.com
Content-Type: application/json
{
    "action": "createUser",
    "userData": {
        "name": "Mario Rossi",
        "email": "mario.rossi@example.com"
    }
}
```

3. **Eliminare un utente:**
   Ancora un POST verso lo stesso endpoint, con `"deleteUser"` specificato nel corpo.

```JSON
POST /useService
Host: api.example.com
Content-Type: application/json
{
    "action": "deleteUser",
    "userId": 456
}
```

In tutti i casi:

- **il server deve leggere il payload e decidere quale funzione eseguire.**
- **L'URL non esprime alcun significato, e i verbi HTTP non aiutano a comprendere l'intenzione dell'operazione.**

- **Tutto è concentrato nel corpo del messaggio, rendendo l'API poco leggibile e meno standard.**

Livello 1 – Risorse (Resources)

Il **Livello 1** rappresenta il **primo, fondamentale passo** per uscire dalla *"palude del POX"* vista nel Livello 0.
In questa fase, l'API smette di essere un insieme confuso di chiamate a funzioni remote e **inizia a organizzare il servizio attorno al concetto di risorsa, che è il pilastro di ogni architettura REST.**

Dal Livello 0 al Livello 1: cosa cambia davvero

Nel Livello 0:

- **esisteva un unico endpoint;**
- **le operazioni erano indicate nel payload;**
- **l'HTTP era usato come semplice "tunnel".**

Nel Livello 1:

- **si abbandona l'endpoint unico;**
- **ogni elemento del dominio viene modellato come risorsa;**
- **ogni [[#Il concetto di Risorsa in REST|risorsa]] ottiene un proprio indirizzo univoco (URI).**

Non si inviano più istruzioni del tipo *"esegui questa operazione"*, **ma si interagisce direttamente con indirizzi che rappresentano ciò su cui si sta operando.**

Il concetto di Risorsa in REST

In un'architettura REST, una **risorsa** non è un metodo, una funzione o un'azione. **È un'informazione, un dato o un concetto del dominio applicativo che il sistema decide di rendere accessibile e manipolabile tramite il web.**

In altre parole:

> **Una risorsa è qualunque informazione significativa del dominio che si vuole rendere nota, consultabile o modificabile da un client.**

Una risorsa può rappresentare:

- **un’entità concreta (un professore, un prodotto, un ordine);**
- **un’entità logica (una sessione d’esame, un corso);**
- **una collezione di entità (tutti i professori);**
- uno stato o una rappresentazione di dati (previsioni meteo, statistiche, report).

Risorsa ≠ Dato fisico ≠ Record di database

È fondamentale chiarire una distinzione concettuale importante:

La risorsa non coincide con il dato fisico memorizzato nel database.

- **il database è un dettaglio implementativo del server;**
- **la risorsa è un’astrazione, ovvero ciò che il client percepisce e utilizza.**

Ad esempio:

- **un professore può essere memorizzato in più tabelle;**
- **può essere ricostruito tramite query complesse;**
- **ma per il client rimane semplicemente “il professore con id 123”.**

REST separa quindi **il modello interno** dalla **rappresentazione esterna**.

Sintesi concetuale

> **Una risorsa è qualunque “cosa” del dominio applicativo che si voglia rendere accessibile tramite il web.**
> Può essere un oggetto fisico, un’entità logica o anche un’informazione astratta.

Esempi tipici di risorse:

- un libro venduto online;
- un utente di un sistema;
- le previsioni meteo di una città;
- un corso universitario;
- un volo aereo;
- un valore di cambio valuta;
- una voce enciclopedica.

> La risorsa diventa quindi **l'elemento centrale** del modello REST.

L'identificatore univoco: l'URI

Affinché una risorsa possa essere utilizzata, deve essere identificabile in modo univoco.
In REST, questo avviene tramite un **URI (Uniform Resource Identifier)**.

> **L'URI è il nome univoco di una risorsa all'interno del web.**

L'URI non descrive l'azione da compiere, ma identifica la risorsa stessa.

La regola fondamentale del Livello 1 del Richardson Maturity Model è:

> **Ogni risorsa deve avere un identificatore univoco, chiamato URI.**

Struttura di un URI

Un **URI (Uniform Resource Identifier):**

- **è l'identificatore univoco di una risorsa nel web.**
  Nel contesto delle API REST, un URI viene quasi sempre espresso come **URL**, perché utilizza il protocollo HTTP per indicare anche dove reperire la risorsa.

Un URI è composto concettualmente da due parti:

- **Parte fissa(URL)**:
  - indica il dominio o il servizio
  - **Indica dove si trova il servizio e quale protocollo utilizzare per accedervi.**
  - **Questa parte identifica il dominio e rende l'URI un URL a tutti gli effetti.**
    - Esempio:
      `https://www.amazon.it`
- **Parte variabile (path dell'URI)**:
  - **Identifica la risorsa specifica all'interno del dominio.**
  - **È la parte semanticamente più importante in REST, perché descrive *che cosa* si sta rappresentando, non *che operazione* si vuole eseguire.**
  - identifica la risorsa specifica

- Esempio:
  - `/professors/123`
    In questo caso:
- `/professors` : **rappresenta la collezione di risorse** .
- `123` : **identifica una singola risorsa all'interno di quella collezione** .
  L'URI completo è:

Plain text

```
https://www.amazon.it/professors/123
```

Identifica quindi in modo univoco **la risorsa "professore con id 123"**, indicando sia *dove* si trova il servizio sia *quale* risorsa è coinvolta.

Concetto chiave

L'URI descrive sempre su cosa si sta operando, non quale azione si vuole compiere.

L'azione (lettura, creazione, aggiornamento, cancellazione) viene invece determinata dal **verbo HTTP** utilizzato (`GET` , `POST` , `PUT` , `DELETE`).

L'URI, da solo, comunica **su cosa** si sta operando, indipendentemente dall'azione che verrà eseguita.

URI e URL: chiarimento fondamentale

Spesso i termini **URI** e **URL** vengono usati come sinonimi, ma **non sono la stessa cosa**.

Definizione corretta

- **[[#**L'identificatore univoco l'URI**|URI (Uniform Resource Identifier)]]**
  - **Identifica una risorsa in modo univoco.**
- **URL (Uniform Resource Locator)**
  - **È un tipo particolare di URI che indica anche dove e come accedere alla risorsa.**

🎟 **Tutti gli URL sono URI, ma non tutti gli URI sono URL.**

☰ Esempi di URI e significato:

Consideriamo un dominio universitario:

- `/professors`
  → elenco di tutti i professori
- `/professors/123`
  → dati del professore con id 123
- `/professors/123/lectures`
  → lezioni tenute dal professore 123
- `/lectures`
  → elenco di tutte le lezioni
- `/lectures/ing_sw`
  → dati del corso di Ingegneria del Software
- `/lectures/ing_sw/examsessions`
  → sessioni d'esame del corso
- `/examsessions/110`
  → dati della sessione d'esame con id 110

Dal punto di vista del server, queste risorse possono essere viste come **oggetti del dominio "esposti" tramite URI**.

Regole e convenzioni per gli URI

Per rendere un'API coerente, leggibile e prevedibile, si seguono alcune best practice:

1. **Nomi al plurale per le collezioni**
   - `/professors`
2. **ID per accedere a una singola risorsa**
   - `/professors/123`
3. **Query string per parametri opzionali o filtri**

Swift

```
`/professors/123?newMail=giordano@uni.na.it`
```

3. **Body della richiesta per dati complessi**
   - **Usato, ad esempio, per inviare un oggetto JSON durante la creazione o modifica di una risorsa.**

Queste regole non sono obblighi formali dello standard HTTP, ma **convenzioni consolidate** che migliorano la chiarezza dell'API.

URI progressivi e risorse correlate

Le risorse sono spesso collegate tra loro da relazioni logiche (esempio: *un professore insegna delle materie*).

Queste relazioni vengono rappresentate direttamente nella struttura dell'URI.

Regola generale

Plain text

```
/risorsaPrimaria/id/risorsaCorrelata
```

Esempio

Per ottenere le materie insegnate dal professore con id 123:

Plain text

```
http://universita.napoli.it/professori/123/materie
```

Questo approccio rende la relazione tra le risorse esplicita, favorendo una struttura gerarchica e intuitiva.

Esempi pratici tra URL e URI

Esempio di URI (non URL):

Plain text

```
urn:isbn:9780132350884
```

- **identifica un libro tramite ISBN;**
- **non dice dove trovarlo;**
- **non usa HTTP.**

Esempio di URL (quindi anche URI):

Plain text

```
https://myUniversity.it/professors/123
```

- **identifica una** risorsa**;**
- **indica il protocollo (** `https` **);**
- **indica dove reperirla (** `myUniversity.it` **).**

Nel contesto REST:

- **si usano quasi sempre URL;**
- **ma concettualmente REST parla di URI, perché il punto centrale è l'identificazione, non il trasporto.**

URI e Risorsa: come leggere correttamente gli esempi

Consideriamo questo URI:

Plain text

```
http://myUniversity.it/professors/123

```

- la **risorsa** è il professore con id 123;
- l'**URI è il nome della risorsa**, non la risorsa stessa.

Quindi:

- `/professors` → risorsa *collezione di professori*
- `/professors/123` → risorsa *professore 123*
- `/professors/123/lectures` → risorsa *insieme delle lezioni del professore 123*

Qui l'id non è la risorsa, ma la identifica:

> **L'ID non è la risorsa, ma è una parte dell'identificatore della risorsa.**

- `123` da solo non è una risorsa;
- `professors/123` identifica una risorsa;
- il significato dell'ID esiste solo **nel contesto della collezione**.

Questo evita ambiguità:

- `123` come professore;
- `123` come corso;
- `123` come esame.

La potenza dei link (URI)

L'uso di **URI ben progettati e standardizzati** è uno degli elementi che rende il Web — e in particolare i sistemi REST — estremamente potenti e flessibili.

Un URI non è soltanto una stringa tecnica, **ma un vero e proprio contratto di comunicazione tra client e server, con diversi vantaggi fondamentali.**

Perché gli URI sono così potenti

Un URI offre benefici su più livelli:

1. **Comprensibilità per gli esseri umani**
   - **Un URI ben strutturato è leggibile e auto-esplicativo.**
     Ad esempio:

```
/users/123
```

fa immediatamente intuire che si sta parlando dell'utente con identificativo `123`.

2. **Interpretabilità da parte delle macchine**
   - **I client (browser, app mobile, servizi backend) sanno che un indirizzo che inizia con `http://` o `https://` rappresenta una risorsa accessibile tramite il protocollo HTTP.**
3. **Standard globale**
   - **Gli URI permettono di identificare e richiamare risorse distribuite su qualsiasi server nel mondo, senza ambiguità, rendendo possibile l'interoperabilità tra sistemi diversi.**
     In altre parole, l'URI risponde sempre alla domanda:

> **"Qual è la risorsa con cui sto interagendo?"**

URI e HTTP: la negoziazione del contenuto

Un aspetto fondamentale è che gli URI sono strettamente legati al protocollo **HTTP**, che fornisce meccanismi avanzati per la gestione della comunicazione, tra cui la **Content Type Negotiation** (negoziazione del contenuto).

Cos'è la Content Type Negotiation

La negoziazione del contenuto consente a **client e server di "parlare la stessa lingua",**
scegliendo il formato più adatto per rappresentare una risorsa.

Non tutte le applicazioni comprendono gli stessi formati:

- **un browser comprende bene l'**HTML**;**
- **molte** API **lavorano con** JSON**;**
- **alcuni sistemi utilizzano XML;**
- **altri possono restituire immagini, video o file binari.**

Come funziona la negoziazione

Il meccanismo si basa sugli **header HTTP**:

1. **Il client chiede**
   Il client specifica, tramite l'header `Accept`, i formati che è in grado di comprendere.

   Esempio:

Plain text

```
Accept: application/json
```

- (il client sta dicendo: "Se possibile, rispondimi in JSON").

2. **Il server risponde**

   - **Il server sceglie uno dei formati supportati e indica il formato effettivo della risposta tramite l'header `Content-Type`.**

   Esempio:

Plain text

```
Content-Type: application/json
```

Un esempio intuitivo è quello dei contenuti multimediali:

se un browser richiede un URI e il server risponde con

Plain text

```
Content-Type: video/mp4
```

il browser sa che non deve visualizzare testo, ma avviare il lettore video integrato.

Gli header HTTP: informazioni aggiuntive

Gli header HTTP accompagnano ogni richiesta e risposta e costituiscono un insieme di metadati che forniscono contesto all'interazione.

Oltre a `Accept` e `Content-Type`, esistono molti altri header importanti, tra cui:

- **Authorization**
  - **Per inviare credenziali (token, API key) e autenticare il client.**
- **Cache-Control**
  - **Per indicare se e per quanto tempo una risposta può essere memorizzata in cache.**
- **User-Agent**
  - **Per descrivere il tipo di client che sta effettuando la richiesta (browser, app mobile, servizio backend).**

In sintesi:

- **l'URI dice *cosa* si vuole**,
- **gli header HTTP specificano *come*, *chi* e *in quale formato***.

Dal Livello 0 al Livello 1: risorse sì, verbi no

A differenza del **Livello 0 (Swamp of POX)**, al **Livello 1** iniziano a comparire **[[#**Struttura di un URI**|URI distinti]] per le** risorse.
Non esiste più un unico endpoint, ma indirizzi specifici:

- **Collezione di utenti:**

Plain text

```
https://api.example.com/users
```

- **Singolo utente:**

Plain text

```
https://api.example.com/users/{userID}
```

Tuttavia, nonostante la migliore modellazione delle risorse, **tutte le operazioni continuano a usare il metodo POST**, senza sfruttare i verbi HTTP in modo semantico.

Esempi di interazione a Livello 1

1. Ottenere gli album di un artista
In un servizio musicale:

- Risorsa: album dell'artista con ID `queen`
- URI:

Plain text

```
https://api.musica.com/artists/queen/albums
```

Interazione: si usa `POST` per ottenere i dati, invece del verbo corretto `GET`.

Plain text

```
POST /artists/queen/albums
Host: api.musica.com
```

2. Aggiungere un prodotto al carello
   In un sito e-commerce:

- **Risorsa**: carrello dell'utente con ID `123`
- **URI**:

Plain text

```
https://api.commerce.com/carts/123/items
```

- **Interazione**:
  si usa `POST` per aggiungere un prodotto (in questo caso è “casualmente” corretto),
  ma anche operazioni di sola lettura potrebbero usare `POST`, perdendo chiarezza semantica.

Plain text

```
POST /carts/123/items
HOST: api.commerce.com
Content-Type: application/json
{
    "productId": "xyz-456",
    "quantity": 2
}
```

3. Cancellare un commento da un Post:
   In un blog:

- **Risorsa**: commento `987` del post `45`
- **URI:**

```
https://api.blog.com/posts/45/comments/987

```

Interazione: si utilizza `POST` per cancellare il commento, invece del verbo corretto `DELETE`.

Plain text

```
POST /posts/45/comments/987
Host: api.blog.com
Content-Type: application/json
{
    "action": "delete"
}
```

Considerazione chiave sul Livello 1

Il Livello 1 rappresenta un **passo avanti importante**:

- **introduce il concetto di risorsa;**
- **sfrutta** URI significativi **e** gerarchici**.**

Tuttavia:

- **==non sfrutta ancora la semantica dei verbi HTTP**==;
- **l'azione è implicita e non chiaramente espressa dal protocollo.**

Questo limite verrà superato nel **Livello 2**, dove i verbi HTTP iniziano finalmente a esprimere il significato dell'operazione.

Livello 2: Verbi HTTP(HTTP Verbs)

Dopo aver introdotto, con il **Livello 1**, il concetto di **risorsa** **identificata da [[#**Struttura di un URI**|URI]] significativi**, il passo successivo verso una vera architettura REST consiste nello sfruttare **in modo corretto i metodi (o verbi) del protocollo HTTP**.

Al **Livello 1** l'API è già strutturata attorno alle risorse, ma **tutte le operazioni vengono ancora effettuate con POST**,
rendendo l'intenzione dell'azione implicita e poco chiara.
Il **Livello 2** supera questo limite:

- **l'azione diventa esplicita, perché è il verbo HTTP stesso a indicare cosa si intende fare sulla risorsa.**

In altre parole:

- **l'URI continua a indicare *su cosa* si opera;**
- **i==l verbo HTTP indica *che cosa* si vuole fare**.==

Come funziona il livello 2

In un'API di Livello 2, ogni operazione utilizza il **verbo HTTP più appropriato**, sfruttando appieno la semantica del protocollo:

- **GET** → leggere una risorsa
- **POST** → creare una nuova risorsa
- **PUT** / **PATCH** → aggiornare una risorsa esistente
- **DELETE** → eliminare una risorsa

Grazie a questa combinazione, l'interfaccia diventa
standard, prevedibile e facilmente comprensibile, anche per chi non

conosce l'implementazione interna del servizio.

Non a caso, **la maggior parte delle API REST "commerciali" si ferma al Livello 2**, poiché rappresenta un eccellente compromesso tra pragmatismo e aderenza ai principi REST.

Il metodo GET: leggere una risorsa

Il metodo **GET** è l'operazione di **sola lettura** per eccellenza.

- **Scopo**:
  - **recuperare la rappresentazione di una risorsa identificata dall'URI.**
- **Caratteristica fondamentale**:
  - **non modifica mai lo stato del server.**

Esempi di utilizzo:

- Ottenere i dettagli di un prodotto in vendita
- Recuperare la lista degli esami sostenuti da uno studente

È importante notare che, anche quando un URI rappresenta un **processo che esegue un calcolo**, una richiesta GET deve restituire **solo il risultato**, senza causare effetti collaterali sul sistema.

Il metodo di POST: creare nuove risorse

Il metodo **POST** serve principalmente per **inviare dati al server** e creare una nuova risorsa.

- **Scopo principale**:
  - **creare una nuova entità come "figlia" della risorsa indicata nell'URI.**
- **Body della richiesta**:
  - **POST può includere un corpo (body), tipicamente in formato JSON, per trasportare dati complessi.**
- **Uso alternativo**:
  - **può essere impiegato per operazioni non riconducibili agli altri verbi o quando PATCH non è supportato.**

Esempi tipici

- Invio dei dati di un form di registrazione

- Pubblicazione di un commento su un blog
- Caricamento di un file (upload)

A differenza di GET, POST modifica lo stato del server a ogni invocazione.

PUT e PATCH: aggiornare una risorsa

Entrambi i metodi servono per **modificare una risorsa esistente**, ma con una differenza concettuale fondamentale.

PUT: aggiornamento totale

- **Il client invia l'intera rappresentazione aggiornata della risorsa.**
- **Se la risorsa esiste, viene completamente sostituita.**
- **Se non esiste, il server può crearla.**

Risposte comuni:

- `200 OK` o `204 No Content` → aggiornamento riuscito
- `201 Created` → risorsa creata

PATCH: aggiornamento parziale

- **Il client invia solo i campi da modificare.**
- **Non richiede l'invio dell'intera risorsa.**
- **Ha esclusivamente semantica di aggiornamento.**

PATCH è quindi più efficiente quando le modifiche sono limitate.

Il metodo DELETE: eliminare una risorsa

Il metodo DELETE serve per rimuovere una risorsa specifica.

- **Scopo**:
  - **eliminare la risorsa identificata dall'URI.**

Risposte comuni

- `200 OK` → eliminazione riuscita con messaggio di conferma
- `202 Accepted` → eliminazione avviata ma non ancora completata
- `204 No Content` → eliminazione riuscita senza corpo di risposta

È importante sottolineare che una risposta positiva indica che la risorsa **non è più accessibile**, non necessariamente che sia stata immediatamente rimossa dal

supporto fisico.

Sicurezza e Idempotenza: concetti chiave del Livello 2

Al Livello 2 diventano centrali due proprietà fondamentali dei metodi HTTP.

Sicurezza (Safety)

Un metodo è sicuro se la sua esecuzione non modifica lo stato del server.

- **Metodo sicuro per definizione**: GET
  - Si possono effettuare infinite richieste GET senza alterare la risorsa.

Idempotenza (Idempotence)

Un metodo HTTP è definito **idempotente** quando:

> **l'esecuzione ripetuta della stessa richiesta produce sempre lo stesso stato finale sul server, indipendentemente dal numero di volte in cui viene inviata.**

È importante sottolineare che:

- **idempotente non significa "non modifica nulla"**;
- significa che **ripetere l'operazione non altera ulteriormente il risultato finale**.

Metodi Idempotenti:

I seguenti verbi HTTP sono idempotenti per definizione:

- **GET**
  - **Recupera una risorsa senza modificarne lo stato.**
  - **Ripetere una richiesta GET non produce alcun effetto collaterale.**
- **PUT**
  - **Aggiorna (o crea) una risorsa sostituendone l'intera rappresentazione.**
  - **Inviare più volte la stessa richiesta PUT porta sempre allo stesso stato finale.**
- **DELETE**
  - **Elimina una risorsa.**

- **Dopo la prima eliminazione, ulteriori richieste DELETE non cambiano più lo stato del server.**

Metodo non idempotente

- **POST**
  - **È utilizzato per creare nuove risorse o avviare operazioni generiche.**
  - **Ogni richiesta POST produce un nuovo effetto, anche se i dati inviati sono identici.**

Esempi:

1. GET – Idempotente (e sicuro)
   Scenario: recuperare i dati di un professore.

```http
GET /professors/123
```

- La risorsa viene solo letta.
- Nessun dato viene creato, modificato o eliminato.

Ripetizione della richiesta:

- 1 volta → ottengo i dati del professore.
- 10 volte → ottengo sempre gli stessi dati.

✓ **Idempotente:**

- **Stato del server invariato.**
- **GET è idempotente e sicuro.**

2. POST – NON idempotente
   - **Scenario:** creare un nuovo commento.

```http
POST /posts/45/comments
```

Body:

JSON

```
{
  "text": "Ottimo articolo!"
}
```

Ripetizione della richiesta:

- 1 volta → viene creato un commento.
- 3 volte → vengono creati **3 commenti identici**, ciascuno con un nuovo ID.

> ✕ **Non Idempotenza**
>
> - **Lo stato del server cambia a ogni richiesta.**
> - **POST NON è idempotente.**

3. **PUT – Idempotente**
   - **Scenario:** aggiornare completamente i dati di un professore.

HTTP

```
PUT /professors/123
```

Body:

JSON

```
{
    "name": "Mario Rossi",
    "email": "m.rossi@uni.it"
}
```

Ripetizione della richiesta:

- 1 volta → il professore viene aggiornato.
- 10 volte → il risultato finale è identico.

> ✓ **Idempotente**
>
> - Anche se **modifica lo stato**, il risultato finale non cambia.

- **PUT è idempotente.**

> **Nota importante:**
>
> Se la risorsa non esiste, PUT può crearla.
> Anche in questo caso, ripetere la richiesta porta sempre allo stesso stato finale.

4. **PATCH – Generalmente NON idempotente**
   - **Scenario:** aggiornare solo l'email del professore

HTTP

```http
PATCH /professors/123
```

Body:

JSON

```json
{
  "email": "nuova.mail@uni.it"
}
```

- Se la modifica è "imposta un valore", può essere idempotente.
- Se la modifica è "incrementa", "aggiungi", "toglie", **non lo è**.
  Esempio non Idempotenza:

JSON

```json
{
  "loginCount": "+1"
}
```

In questo caso:

- Il risultato cambia a ogni invio.
- **PATCH non garantisce l'idempotenza per definizione.**

5. **DELETE – Idempotente**
   - **Scenario:** eliminare un professore.

```http
DELETE /professors/123
```

Ripetizione della richiesta:

- 1 volta → la risorsa viene eliminata.
- 5 volte → la risorsa è comunque eliminata.

✓ Idempotenza

Dopo la prima richiesta, lo stato finale non cambia più.
DELETE è idempotente.

> Anche se il server risponde con errori o status diversi, **l'effetto finale è lo stesso**.

🎟 Concetto Chiave:

Idempotenza non significa "non modifica"
Significa **"ripetere la stessa richiesta non cambia il risultato finale"**.

Verbo	Modifica lo stato	Idempotente	Sicuro
GET	❌ No	✅ Sì	✅ Sì
POST	✅ Sì	❌ No	❌ No
PUT	✅ Sì	✅ Sì	❌ No
PATCH	✅ Sì	⚠️ Dipende	❌ No
DELETE	✅ Sì	✅ Sì	❌ No

PUT vs POST: la differenza concettuale

La differenza tra **PUT** e **POST non riguarda semplicemente il fatto che entrambi "inviano dati"**, **ma chi determina l'identità della risorsa e che tipo di effetto produce la richiesta sullo stato del server**.

1. **PUT: imporre uno stato finale**
   - Con **PUT**, il client sta dicendo al server:

        - ***“Questa risorsa deve essere esattamente così.”***
    - Caratteristiche fondamentali:
        - **Il client conosce già l’URI della risorsa**
        - **L’URI identifica direttamente** la risorsa da creare o aggiornare
        - Il **body contiene la rappresentazione completa dello stato finale**
        - Ripetere la stessa richiesta **non modifica il risultato finale**
        - **PUT è idempotente**
        - Serve a ottenere **uno stato finale prevedibile**

HTTP

```http
PUT /users/123
```

JSON

```json
{
  "id": 123,
  "name": "Marco",
  "email": "marco@email.it"
}
```

Comportamento del server:

- Se `/users/123` **esiste** → viene **completamente aggiornato**
- Se `/users/123` **non esiste** → viene **creato**
- Se la richiesta viene inviata più volte → **lo stato finale rimane invariato**

> ⓘ **Perché PUT è idempotente?**
>
> Perché il client **impone uno stato finale**, invece di richiedere l’esecuzione di un’azione.

POST: eseguire un’azione o creare una nuova identità

- Con **POST**, il client sta dicendo al server:
  - “Esegui questa operazione con questi dati.”
    - Caratteristiche fondamentali:
        - Il client **non conosce l’URI finale della risorsa**
        - L’URI rappresenta una **collezione** o un **processo**

- Il server decide **come e dove** creare la risorsa
- Ogni richiesta produce **un nuovo effetto**
- **POST non è idempotente**
- Ogni invocazione rappresenta **una nuova azione**

Esempio:

HTTP

```http
POST /users
```

JSON

```json
{
  "name": "Marco",
  "email": "marco@email.it"
}
```

- Comportamento del server:
- Prima richiesta → `/users/124`
- Seconda richiesta → `/users/125`
- Terza richiesta → `/users/126`

Ogni chiamata POST crea **una nuova risorsa distinta**.

ⓘ Perché POST non è idempotente?

Perché ogni richiesta **aggiunge qualcosa di nuovo** allo stato del server.

La distinzione fondamentale

La vera differenza può essere riassunta così:

PUT lavora su una risorsa con identità già nota al client.
POST delega al server la creazione di una nuova identità.

Comprendere questa distinzione è essenziale per progettare API **coerenti, robuste, prevedibili e realmente RESTful**.

Sicurezza e Idempotenza: perché sono centrali nel Livello 2

Nel **Livello 2 della Richardson Maturity Model**, l'uso corretto dei verbi HTTP non serve solo a rendere l'API più leggibile, ma introduce anche **proprietà fondamentali del protocollo HTTP** che hanno un impatto diretto su affidabilità, robustezza e gestione degli errori.

Le due proprietà più importanti sono:

- **Sicurezza (Safety)**
- **Idempotenza (Idempotence)**

Comprenderle è essenziale per progettare API REST coerenti e prevedibili.

Sicurezza (Safety)

Un metodo HTTP è definito **sicuro** quando:

la sua esecuzione non modifica lo stato del server.

Il caso più significativo è il metodo **GET**.

- **GET è sicuro per definizione**
- **Serve esclusivamente a leggere una risorsa**
- **Può essere invocato più volte senza alcun effetto collaterale**

Questa caratteristica è ciò che rende possibili funzionalità fondamentali del web, come:

- cache,
- bookmark,
- prefetch automatico,
- retry automatici da parte dei client o dei proxy.

Idempotenza (Idempotence)

Un metodo HTTP è **idempotente** quando:

ripetere più volte la stessa richiesta produce sempre lo stesso stato finale sul server.

È importante sottolineare che:

- **idempotente non significa "non modifica lo stato"**;

- **significa che ripetere l'operazione non cambia ulteriormente il risultato finale.**

Riepilogo delle proprietà dei principali verbi HTTP

Alla luce di queste definizioni, possiamo riassumere il comportamento dei principali metodi HTTP:

- **GET**
  - Sicuro
  - Idempotente
  - Operazione di sola lettura
- **PUT**
  - Non sicuro (modifica lo stato)
  - Idempotente
  - Impone uno stato finale alla risorsa
- **DELETE**
  - Non sicuro
  - Idempotente
  - Dopo la prima eliminazione, lo stato finale rimane invariato
- **PATCH**
  - Non sicuro
  - L'idempotenza **dipende dal tipo di operazione**
  - Non è garantita per definizione
- **POST**
  - Non sicuro
  - **Non idempotente**
  - Ogni invocazione produce un nuovo effetto sul server

⍰ Perché l'idempotenza è così importante nella pratica

L'idempotenza **non è un concetto teorico o accademico**, ma rappresenta un **vantaggio operativo concreto**, soprattutto nei **sistemi distribuiti**, dove errori di rete, timeout e interruzioni di comunicazione sono eventi normali.

Proprio per questo motivo, l'idempotenza rende le API **più robuste, più sicure e più facili da integrare**.

☰ Scenario Tipico

Immaginiamo questo scenario:

Un client invia una richiesta **PUT** per aggiornare una risorsa, ma a causa di un problema di rete **non riceve la risposta**.

Il client si trova in una situazione di incertezza:

- la richiesta è arrivata al server?
- il server l'ha già elaborata?
- oppure la richiesta non è mai stata ricevuta?

Se l'operazione **non fosse idempotente**, il client non saprebbe come comportarsi: ripetere la richiesta potrebbe causare duplicazioni o errori.

Grazie all'idempotenza, invece:

- **il client può inviare nuovamente la stessa richiesta senza alcun rischio;**
- **se la prima richiesta era andata a buon fine, la seconda non cambierà lo stato finale**;
- **se la prima non era arrivata, la seconda completerà correttamente l'operazione**.

In entrambi i casi, **lo stato finale del server sarà quello corretto**.

L'idempotenza

1. **semplifica la logica dei client,**
2. **rende le integrazioni più affidabili**
3. **permette di gestire in modo sicuro le incertezze tipiche delle comunicazioni di rete.**
   Per questo motivo è uno dei concetti chiave nell'uso corretto dei verbi HTTP e nel raggiungimento del **Livello 2 della Richardson Maturity Model**.

In sintesi

- **Livello 1** → introduce le **risorse** e gli **URI significativi**, ma non sfrutta i verbi HTTP.
- **Livello 2** → mantiene le risorse e aggiunge l'uso corretto dei **metodi HTTP**, rendendo l'API espressiva e standard.

Livello 3 – Controlli Ipermediali (HATEOAS)

La "Gloria di REST"

Il **Livello 3** rappresenta il **livello più alto di maturità** nella Richardson Maturity Model
ed è ciò che, secondo l'interpretazione più rigorosa, consente di definire un'API **veramente RESTful**.

A questo livello, REST esprime pienamente la propria filosofia architetturale.

Il concetto chiave su cui si fonda il Livello 3 è **HATEOAS** (*Hypermedia as the Engine of Application State*).

Cos'è HATEOAS

Con HATEOAS, **la risposta del server non si limita a restituire i dati richiesti, ma include anche i controlli ipermediali (link) che indicano quali azioni possono essere eseguite successivamente su quella risorsa.**

Lo stato dell'applicazione non è più "codificato" nel client, ma viene guidato dinamicamente dalle risposte del server.

In altre parole:

- **il server guida il client nel flusso dell'applicazione;**
- **il client non deve conoscere in anticipo tutti gli** endpoint**disponibili;**
- **il percorso operativo emerge dinamicamente, seguendo i link forniti dal server.**

Analogia con il Web

Per comprendere il Livello 3, è utile pensare a come navighiamo normalmente un sito web.

Quando visiti una pagina web:

- **non conosci in anticipo** tutti gli URL delle pagine successive;
- **non devi memorizzarli o “hardcodarli”**;
- è la **pagina stessa** che ti fornisce i link su cui puoi cliccare per proseguire la navigazione.

La navigazione avviene quindi **seguendo collegamenti ipertestuali**, non tramite una conoscenza preventiva della struttura interna del sito.

HATEOAS applica **esattamente questo stesso principio** alle API REST.

Esempio di risposta HATEOAS:

Supponiamo di richiedere le informazioni di un ordine specifico (`GET /ordini/789`).
Oltre ai dati dell’ordine, il server restituisce anche i **link alle azioni consentite in quello stato**:

JSON

```json
{
    "id": 789,
    "stato": "in elaborazione",
    "totale": 99.50,
    "_links": {
        "self": {"href": "/ordini/789"},
        "modifica": {"href":"/ordini/789/modifica"},
        "annulla": {"href": "/ordini789/annulla"}
    }
}
```

Il client **non deve conoscere in anticipo** come modificare o annullare un ordine: **scopre le azioni disponibili direttamente dalla risposta del server**, seguendo i link forniti.

Esempio applicativo segreteria universitaria e HATEOAS

Torniamo all’esempio della **segreteria universitaria** per chiarire il funzionamento pratico di HATEOAS.

Supponiamo che un client richieda **tutte le sessioni d’esame** disponibili per una determinata materia.

In un sistema REST di **Livello 3**, il server **non è obbligato a restituire immediatamente tutti i dettagli completi** di ogni sessione.

Al contrario, può scegliere un approccio più efficiente:

- **fornire un elenco sintetico delle sessioni;**
- **includere, per ciascuna sessione, i link alle azioni disponibili.**

Questo approccio ha un vantaggio importante:
riduce il carico iniziale sulla rete e delega al client la scelta di quali informazioni approfondire.

Controlli ipermediali forniti dal server

Per ogni sessione d'esame, la risposta potrebbe includere link che permettono di:

- ottenere i **dettagli completi della sessione**;
- recuperare le **informazioni sull'aula associata**;
- **prenotare** la sessione d'esame.

Il server non sta solo restituendo dati, ma sta **descrivendo le possibilità di interazione** a partire dallo stato corrente della risorsa.

Risposte auto-descrittive

Se il client decide di seguire uno di questi link (ad esempio per visualizzare i dettagli di una sessione), la risposta successiva del server sarà strutturata in modo **auto-descrittivo**.

Questo significa che:

- **i dati restituiti sono accompagnati dai link alle azioni successive;**
- **il client può continuare a interagire senza conoscere in anticipo la struttura dell'API.**

JSON

```
{
    "id": 123,
    "sessions": [
        {
            "self": "http://myUniversity.ac.uk/examsession/110",
// Link alla risorsa stessa
```

```
            "room": {"self":"http://myUniversity.ac.uk/rooms/222"}, //link alla risorsa aula
            "link":{
                "rel": "http://myUnivesity.ac.uk/examSessions/reserve", //Descrive la relazione (prenotazioni)
                "uri": "/examsession/110" //Link all'azione di prenotazione
            }
        },
        {
            "self": "http://myUniversity.ac.uk/examsessions/221",
            "room": {"self": "http://myUniversity.ac.uk/rooms/115"},
            "link": {
                "rel": "http://myUniversity.ac.uk/examSessions/reserve",
                "uri": "/examsessions/211"
            }
        }
    ]
}
```

Significato architetturale

In questo modello:

- il client **non deve sapere a priori** come si prenota un esame;
- non deve conoscere l'URL o il metodo da usare;
- **scopre tutto dinamicamente seguendo i link presenti nella risposta del server**.

Questo è il cuore di HATEOAS:
il flusso dell'applicazione emerge dalle risposte, non dal codice del client.

> ⓘ **Chiarimento fondamentale sul livello 3 (HATEOAS)**
>
> In un'architettura REST che implementa il **Livello 3** della Richardson Maturity Model:
>
> - **non è il client** a decidere o "costruire" autonomamente i link delle azioni successive;

- **non è il JSON inviato dal client** a definire i percorsi operativi;
- **è il server** che, in base allo **stato corrente della risorsa**, comunica al client **quali azioni sono possibili** e **come eseguirle**.

🔗 Cosa cambia rispetto ai livelli precedenti

Nei livelli **1 e 2**:

- il client **conosce già** gli endpoint;
- gli URL sono spesso **hardcoded** nel codice del client;
- il flusso applicativo è deciso **a priori dal client**.

☰ Esempio tipico (Livello 2):

- il client "sa" che per annullare un ordine deve chiamare

HTTP

```
DELETE /ordini/{id}
```

🔗 Cosa succede nel livello 3 (HATEOAS)

Nel **Livello 3**:

- il client **non deve conoscere** in anticipo:
  - **gli** endpoint**;**
  - **la struttura degli URL;**
  - **quali azioni siano consentite in uno specifico stato.**
- il server risponde con:
- **i dati della risorsa;**
- **i link ipermediali alle azioni consentite in quello stato.**

Quindi:

È il server a "dire al client cosa può fare dopo".

Quindi il client cambia completamente ruolo:

- **non costruisce URL;**
- **non decide il flusso applicativo;**
- **segue i link forniti dal server.**

Il client diventa un **interprete di ipermedia**, non un orchestratore della logica.

> 🎟 In un sistema REST di Livello 3, **il client non definisce il flusso dell'applicazione: lo scopre dinamicamente dalle risposte del server**, seguendo i link HATEOAS.

✓ Vantaggi dell'implementazione del livello 3(HATEOAS)

L'adozione del **Livello 3** nel modello di maturità REST non è solo una scelta stilistica, ma introduce **vantaggi architetturali concreti**, soprattutto nei sistemi complessi e destinati a evolvere nel tempo.

1. **Disaccoppiamento tra Client e Server**

Questo è il **vantaggio più rilevante** del Livello 3.

In presenza di HATEOAS,
il client non deve più conoscere né memorizzare la struttura degli endpoint **dell'API.**
Gli URL non sono hardcoded nel codice del client, ma vengono scoperti dinamicamente seguendo i link forniti dal server.

Di conseguenza:

- **il server può modificare la struttura degli URL (ad esempio da** `/reserve` **a** `/booking/new`**);**
- **il client continua a funzionare senza modifiche, perché si limita a seguire i nuovi link ricevuti;**
- **client e server possono evolvere in modo indipendente, purché il significato dei link rimanga coerente.**

Questo porta a un **disaccoppiamento forte** tra le due parti e riduce drasticamente il rischio di rompere le integrazioni durante l'evoluzione del sistema.

2. Riduzione del Carico di Rete

Un secondo vantaggio importante riguarda l'**efficienza della comunicazione**.

In un sistema che implementa HATEOAS:

- **il server non è obbligato a restituire subito tutti i dettagli di una risorsa;**
- **può invece inviare una rappresentazione più leggera, arricchita da link verso informazioni di dettaglio o azioni successive.**

In questo modo:

- **il client riceve inizialmente solo ciò che è strettamente necessario;**
- **può decidere in modo esplicito se e quando recuperare dati aggiuntivi, seguendo i link forniti;**
- **si evita di trasferire grandi quantità di dati che potrebbero non essere utilizzate.**

Il risultato è una comunicazione più **efficiente**, **scalabile** e meglio adattata alle reali esigenze del client.

Conclusione: REST come Architettura del Web

In definitiva, lo stile **REST**, che raggiunge la sua espressione più completa nel **Livello 3 con HATEOAS**, promuove un'architettura che **riproduce fedelmente il modello del World Wide Web**.

Non si tratta solo di un insieme di regole tecniche, ma di un approccio architetturale fondato su **principi consolidati**, che hanno permesso al web di crescere in modo scalabile, interoperabile e affidabile.

I principi fondamentali su cui si basa un'architettura RESTful sono i seguenti.

1. Separazione delle responsabilità (Separation of Concerns)

In un sistema RESTful, **client e server hanno ruoli chiaramente distinti**:

- **il client è responsabile della presentazione dei dati e dell'interazione con l'utente;**
- **il server si occupa esclusivamente della logica di business, della gestione delle risorse e dei dati.**

Questa separazione riduce l'accoppiamento tra le parti e rende entrambi i componenti:

- più semplici da sviluppare;
- più facili da mantenere;
- più indipendenti nell'evoluzione.

2. Elaborazione intermedia (Intermediary Processing)

REST impone che i messaggi siano **auto-descrittivi**:

- **ogni richiesta è [[#Vincolo "Stateless" (Senza Stato) e Risorse|stateless]];**
- i metodi HTTP e i media type sono standardizzati;
- il significato di una richiesta o di una risposta è comprensibile senza contesto aggiuntivo.

Questa caratteristica permette a **componenti intermedi** — come proxy, gateway, load balancer e cache — di:

- interpretare correttamente i messaggi;
- ottimizzare le prestazioni;
- migliorare la sicurezza e la scalabilità del sistema,

senza che client e server debbano conoscere l'esistenza di tali componenti.

3. Proprietà fondamentali ereditate dal Web

Le architetture RESTful **ereditano direttamente le proprietà che hanno reso il web un'infrastruttura di successo**, tra cui:

- **semplicità:**
  - **grazie all'uso di protocolli e meccanismi standard;**
- **scalabilità:**
  - **intesa come capacità di gestire un numero crescente di utenti e richieste;**
- **portabilità:**
  - poiché client e server possono essere implementati con tecnologie diverse;
- **prestazioni elevate:**
  - favorite da caching, statelessness e distribuzione del carico.

Sintesi finale

Seguire lo stile REST — e in particolare il **Livello 3 con HATEOAS** — significa progettare sistemi che **non solo funzionano**, ma che **possono crescere, evolvere e adattarsi nel tempo**, proprio come il web stesso.

REST non è quindi una moda o una semplice convenzione: **è un modello architetturale maturo, nato dall'osservazione diretta di ciò che ha reso Internet scalabile e sostenibile su scala globale.**